

LLM Inference Optimization

From Token to Scale

What is happening behind every single LLM call?

Inference Now Dominates Total Compute

80-
90%

compute cost is
spent on inference
in production.

- **Training is a Capital Event:** Pre-training is a massive, one-time capital expense. It is computationally brutal but bounded.
- **Inference is an Operating Expense:** Inference is an unbounded, recurring operational cost that runs continuously and scales directly with usage.
- **The Agentic Multiplier:** While per-token costs are falling rapidly, total token volume is exploding. Agentic workflows and "Test-Time Compute" (reasoning models) consume 5–30× more tokens per task.

**“The inflection point for
inference has arrived.”**

Jensen Huang
NVIDIA GTC Keynote 2026



The Three Core Metrics of LLM Inference

TTFT

TIME TO FIRST TOKEN

The latency between the user sending a prompt and the first character appearing on screen.

TPOT / ITL

Time per Output Token
Inter Token Latency

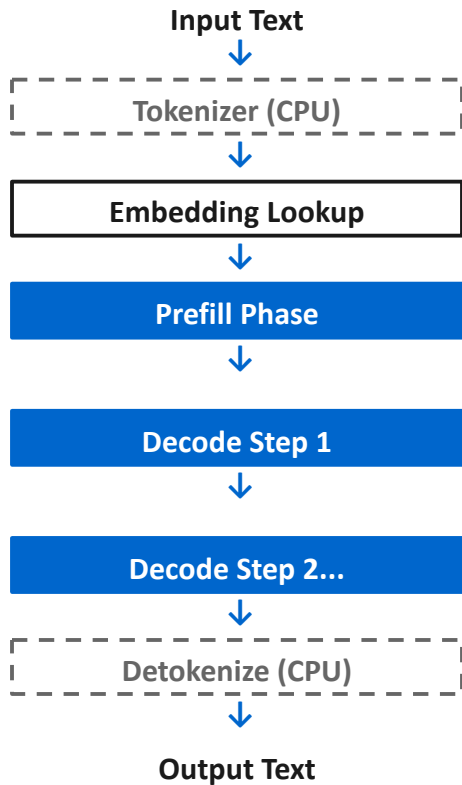
The average time between consecutive output tokens during generation.

Throughput

TOKENS PER SECOND

The total number of tokens generated across all concurrent users per second.

The Inference Pipeline: From Input text to Output text



"Hello, how are you?" → [15496, 11, 703, 389, 345, 30]

token IDs → matrix of shape [seq_len × d_model]

- Single Forward Pass over all prompt tokens
 - Q, K, V projections & Causal self-attention
 - Save all K and V vectors → **KV Cache**
 - Project last token's output → logits → sample token₁
 - Forward Pass over token₁ only
 - Append K, V to KV Cache
 - Attend over full KV Cache (all prior tokens)
 - Project → logits → sample token₂
- KV Cache grows by one entry, repeat until EOS token is sampled ...

token IDs → output string

(streamed token by token to the client)

The tokenizer converts text to integers on the **CPU** before the GPU sees anything

"Hello, how are you?"



15496

11

703

389

345

30

- **Algorithm:** Uses subword tokenization like Byte-Pair Encoding (BPE) or SentencePiece, with vocabularies typically spanning 32k–128k entries.
- **Resource Profile:** Fast, stateless, and involves no floating-point arithmetic. It runs entirely on the CPU.
- **Optimization:** In production systems, tokenization is pipelined with the previous request's decode phase so it never blocks GPU execution.

Tokenizer



Embedding



Prefill

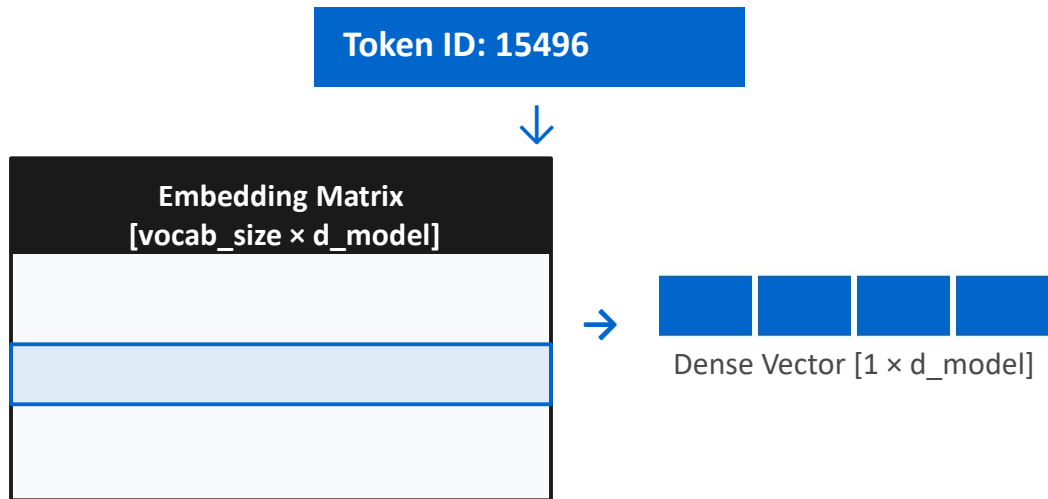


Decode



Detokenize

Token IDs become vectors via a gather operation in GPU HBM



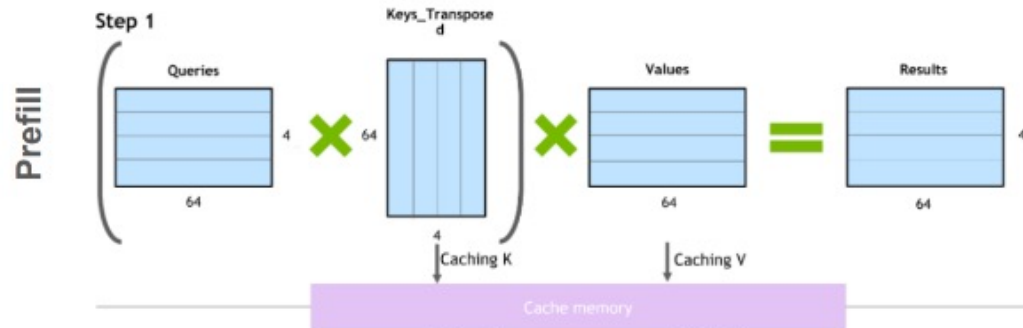
COMMON MISCONCEPTION

Embedding lookup is **NOT** a CPU operation. The embedding matrix lives in GPU memory - **High Bandwidth Memory (HBM)** alongside all other model weights. The gather operation happens entirely on-device to avoid slow data transfers between host RAM and HBM.

Nvidia A100, H100, Blackwell GPUs all using HBM for massive AI training/inference.

Prefill processes all prompt tokens in parallel

$(Q * K^T) * V$ computation process with caching



- **Single Forward Pass:** The entire input sequence is passed through the transformer stack simultaneously.
- **Compute-Bound:** Large matrix multiplications across all tokens saturate the GPU's tensor cores.
- **Transformer Operations:** Self-Attention ($Q \cdot K^T$, causal mask, softmax, weighted V sum), Feed-Forward Networks, and Residuals are applied per token.
- **Output Generation:** The last token's output is projected to normalized probabilities, from which the first output token is sampled.

KEY METRIC IMPACT

Prefill dominates TTFT (Time to First Token).

Tokenizer



Embedding



Prefill



Decode



Detokenize

Every prefill step saves K and V vectors for every token in every layer —the **KV Cache**

Prefill Phase

Processes all prompt tokens



[K, V] Vectors

Per layer, per token



KV Cache

Stored in GPU HBM

- **Efficiency:** The KV Cache is the memory structure that makes the decode phase efficient by avoiding recomputation of past tokens.
- **Persistence:** It must persist in GPU High Bandwidth Memory (HBM) for the entire lifetime of the request.

Tokenizer

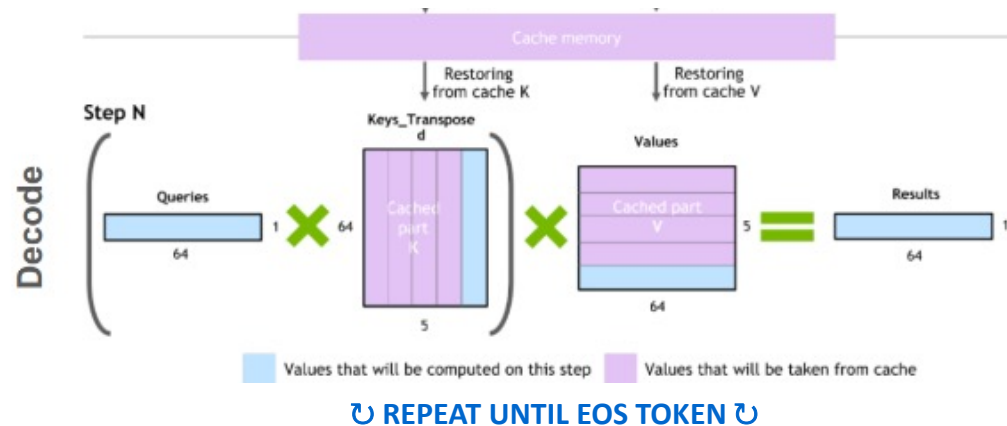
→ Embedding

→ Prefill

→ Decode

→ Detokenize

Decode generates one token at a time



- **KV cache Growth:** It grows by exactly one entry (one K and one V vector per layer) for each new token generated during the decode step.
- **The Bottleneck:** Each step must load the full model weights (~140 GB for a 70B model) and the full KV Cache from HBM to compute the output for just one token.
- **Idle Compute:** Throughput is bounded by **HBM bandwidth** (TB/s, indicating how fast massive data stream into the processor).
- **Metric Impact:** This phase dominates **ITL (Inter-Token Latency)**, determining how smoothly the text streams to the user.

Prefill is **Compute-Bound**. Decode is **Memory-Bound**.

PHASE	BOTTLENECK	SCALES WITH	METRIC DOMINATED
Prefill	Compute (FLOPs)	Prompt length ²	TTFT (Time to First Token)
Decode	Memory Bandwidth	Model size × steps	ITL (Inter-Token Latency)

THE CORE INSIGHT

This fundamental distinction in resource profiles drives every optimization in modern LLM inference engines.

TTFT measures how quickly the first token arrives — it is dominated by **prefill computation**

TIME TO FIRST TOKEN (TTFT)

The total latency from when the API receives a request to when the first generated token is streamed back to the client.

CHAT APPLICATIONS

< **500ms**

CODE COMPLETION

< **200ms**

- **The Long Prompt Penalty:** Because prefill scales with $O(\text{seq_len}^2)$, long prompts (e.g., 8,000-token agentic context windows) directly and quadratically increase TTFT.

ITL measures the gap between successive tokens — it determines whether streamed text feels **fluid**

INTER-TOKEN LATENCY (ITL)

The average time between consecutive output tokens during the decode phase. It is dominated by memory bandwidth and active batch size.

STREAMING TARGET

< 50ms

~20 tokens/sec (faster than human reading speed)

THE CENTRAL TRADE-OFF: ITL v.s. Throughput

Increasing **batch size** improves overall throughput (amortizing the cost of loading weights), but **increases ITL** because each decode step takes longer to process the larger batch.

KV Cache memory management:

The KV Cache exhausts GPU memory before compute does

For a 70B model, just **20 concurrent requests** consume the entire 80 GB HBM of a single H100 GPU. Here is the exact math breakdown:

Model Specs (70B FP16)	32 layers, 64 heads, d_head=128	
KV Cache per token per layer	$2 \times 64 \text{ heads} \times 128 \text{ d_head} \times 2 \text{ bytes}$	= 32 KB
KV Cache for a 4,096-token request	$32 \text{ KB} \times 32 \text{ layers} \times 4,096 \text{ tokens}$	≈ 4 GB

At 20 concurrent requests: 20 requests × 4 GB

= 80 GB

(Entire H100 HBM)

along with LLM weights: 140GB

KV Cache memory management:

Pre-allocating contiguous memory wastes GPU memory

max token output: 4096

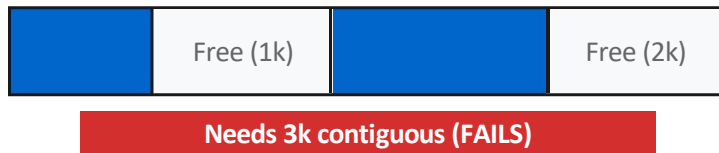
INTERNAL FRAGMENTATION

A short 60-token response still holds a **4,096-token reservation** for its entire lifetime. The unused portion cannot be utilized by any other request.



EXTERNAL FRAGMENTATION

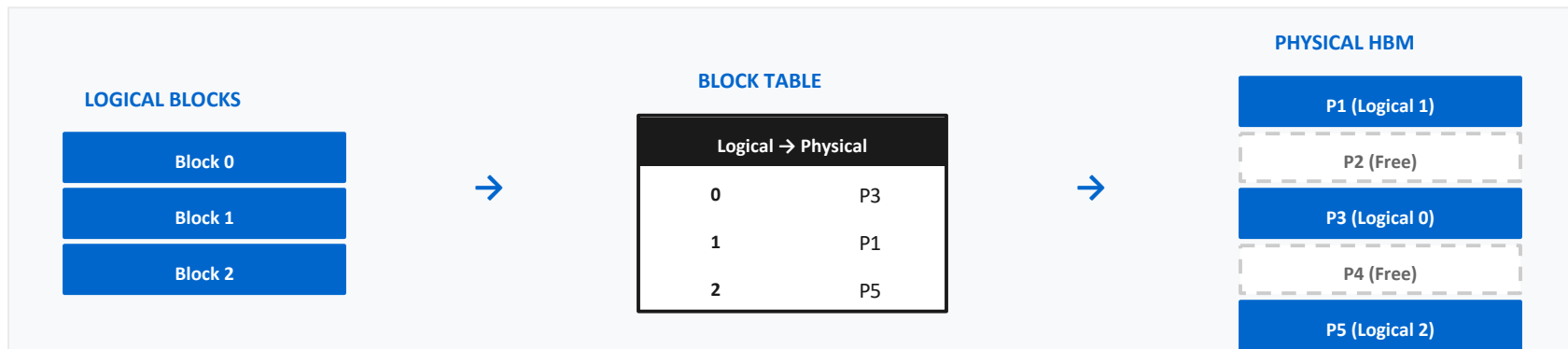
As requests complete, free memory becomes scattered in non-contiguous chunks. A new request needing 2,000 contiguous tokens **fails** even when 3,000 total free tokens exist.



The Result: The system exhausts GPU memory and throws Out-Of-Memory (OOM) errors while compute utilization remains extremely low.

KV Cache memory management:

PagedAttention eliminates fragmentation

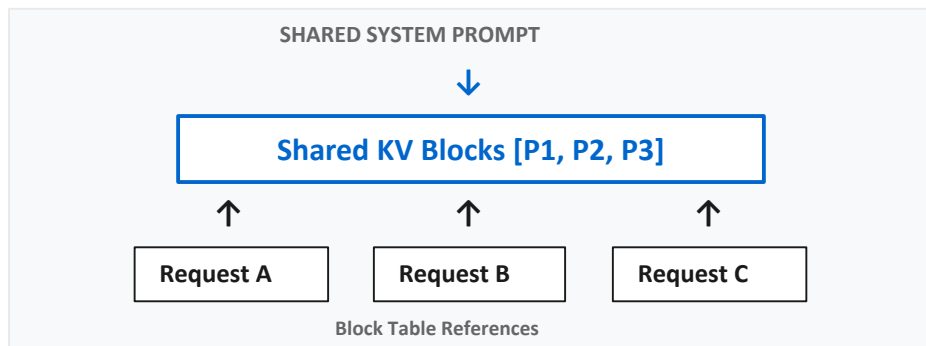


- **OS Inspiration:** Inspired by virtual memory paging (vLLM, SOSP 2023), it decouples logical memory from physical memory.
- **On-Demand Allocation:** Blocks (e.g., 16 tokens) are allocated only as tokens are generated. No contiguous allocation is required.
- **Zero Fragmentation:** Internal fragmentation is limited to a single partial block. External fragmentation is eliminated entirely.

Result: 2–4× higher throughput on the same hardware by maximizing memory utilization.

KV Cache memory management: KV reuse

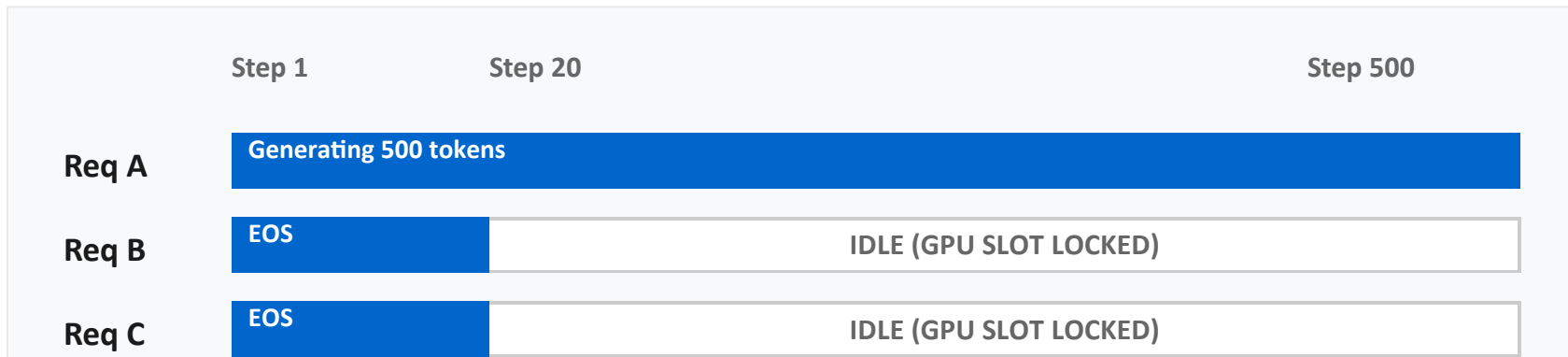
When multiple requests share a system prompt, PagedAttention **computes KV blocks once** and reuses them



- **Reference Counting:** Shared blocks are reference-counted, so the shared prefix (e.g. same system prompt) is not recomputed.
- **Dual Benefit:** Lowers TTFT and reduces GPU memory pressure.
- **Real-World Impact:** Moonshot AI sees >90% KV cache hits on coding workloads for their 2.8T model.

Throughput improvement with Batching

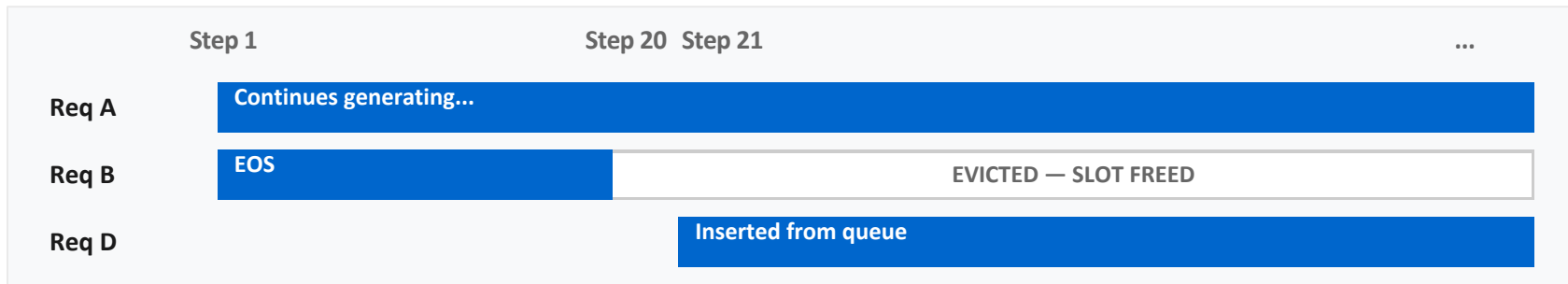
Static batching locks GPU slots until the [slowest request finishes](#), unacceptable for live traffic



- **Wasted Compute:** GPU slots B and C sit completely idle for 480 steps. GPU utilization is artificially capped by the longest request in the batch.
- **Head-of-Line Blocking:** Every queued user must wait for the entire batch to drain before their request can even begin processing.
- **Live Traffic Impact:** This creates massive, unpredictable spikes in TTFT (Time to First Token) for new requests, violating strict latency SLOs.

Throughput improvement with Continuous batching

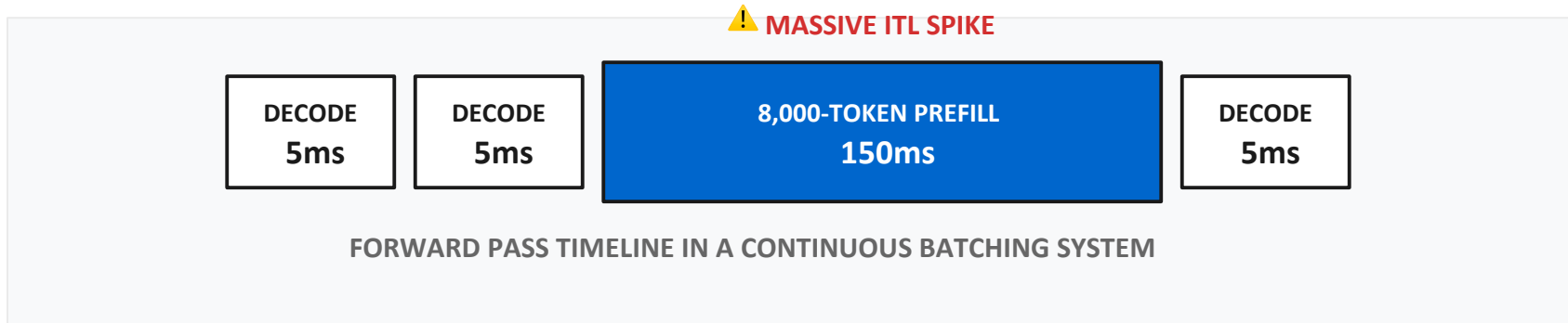
Continuous batching evicts finished requests and inserts new ones after **every single forward pass**.



- **Iteration-Level Scheduling:** After every decode step, the scheduler inspects the batch. EOS requests are evicted; queued requests are inserted immediately.
- **Maximum Utilization:** GPU utilization is bounded only by available KV Cache memory, not by the longest request in the batch.
- **Industry Standard:** This is the baseline scheduling algorithm implemented in every modern inference framework (vLLM, TGI, TensorRT-LLM, SGLang).

Prefill-Decode Interference Problem

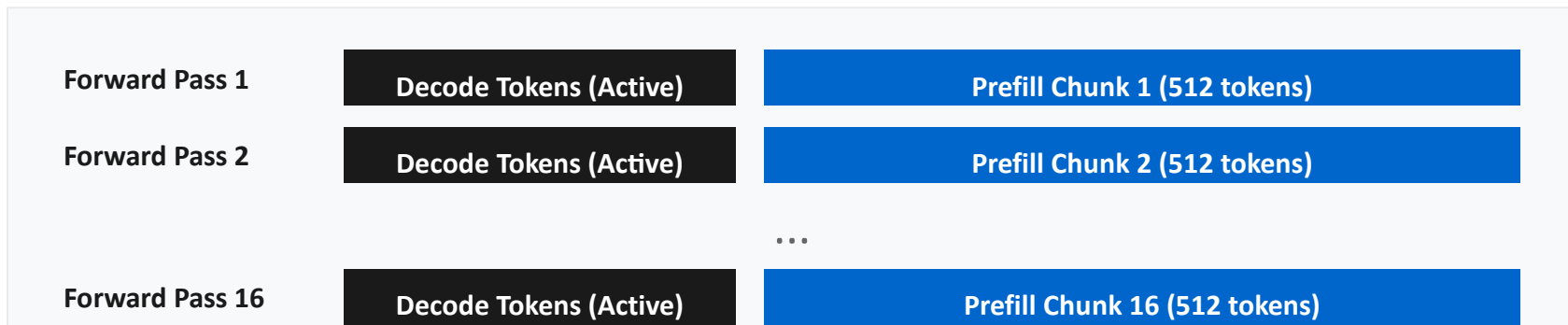
When a long-context request is inserted, its prefill **stalls every concurrent decode step**



- **Quadratic Scaling:** Prefill computation scales with $O(\text{seq_len}^2)$. An 8,000-token prompt can take 100–200ms to process in a single forward pass.
- **Impact on Live Users:** Every other live user in the same batch experiences an Inter-Token Latency (ITL) spike equal to the prefill duration, violating strict streaming SLOs.
- **The Core Challenge:** This is known as the **Prefill-Decode Interference Problem** — the hardest scheduling challenge in live-traffic serving.

Prefill-Decode Interference Problem: Fixed-sized prefill chunks

Chunked prefill interleaves fixed-size prefill chunks with decode steps to enforce a [latency ceiling](#)



- **Mechanism:** An 8,000-token prompt chunked at 512 tokens requires 16 forward passes before the first output token is generated.
- **The Trade-off:** TTFT for the new request increases modestly, but ITL for all concurrent users stays strictly within SLO because no single forward pass is dominated by a massive prefill.
- **Use Case:** The best choice when the cluster lacks high-speed cross-node interconnects (like InfiniBand-Internode connection) required for full disaggregation of prefill/decode.

Prefill-Decode Interference Problem: Separate GPU pools

Routing prefill and decode to dedicated GPU pools provides **complete ITL isolation**



NVLink (Intra-node): NVIDIA's proprietary, high-speed bidirectional interconnect technology to wire multiple GPUs inside a single server node.

- **Hard SLO (Service Level Objective) Isolation:** Prefill workers process full prompts without decode constraints. The decode pool's ITL is completely isolated from prefill load, ensuring smooth streaming for all users.
- **Independent Scaling:** Each pool scales independently. If prompt lengths grow, add prefill nodes. If output lengths grow, add decode nodes.
- **The Trade-off & Routing:** Introduces KV cache transfer cost and system complexity.
- **Routing heuristic:** Disaggregate only long, uncached prompts. Run prefill locally for short or cache-hit requests.

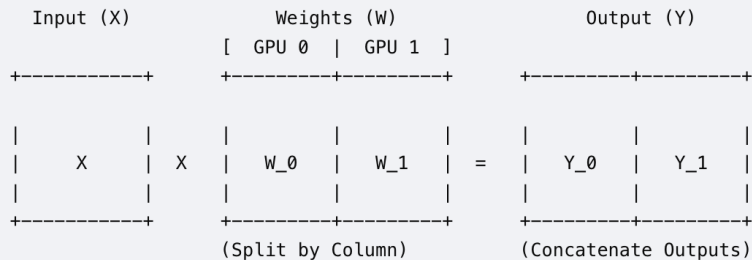
Nvidia NVLink Evolution

Generation	GPU Architecture	NVLink Bandwidth (Bidirectional)	On-Chip HBM Bandwidth (Local)	Primary Memory Type
NVLink 1 & 2	Pascal / Volta (V100)	Up to 300 GB/s	Up to 900 GB/s	HBM2
NVLink 3	Ampere (A100)	600 GB/s	Up to 2.0 TB/s	HBM2e
NVLink 4	Hopper (H100)	900 GB/s	3.35 TB/s	HBM3
NVLink 5	Blackwell (B200)	1.8 TB/s	8.0 TB/s	HBM3e
NVLink 6	Rubin (R100)	3.6 TB/s	22.0 TB/s	HBM4

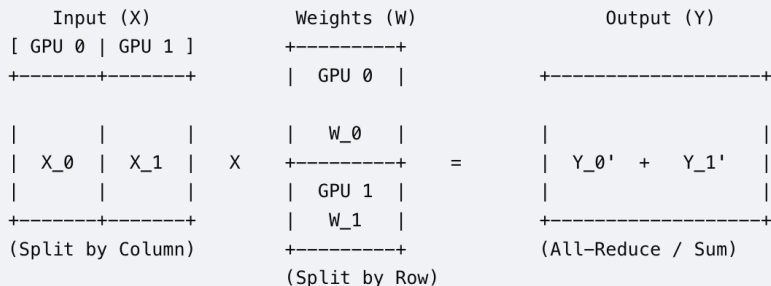
Parallelism: Tensor parallelism

Tensor parallelism shards weight matrices across GPUs, scaling both **memory capacity** and **decode bandwidth**

column parallelism



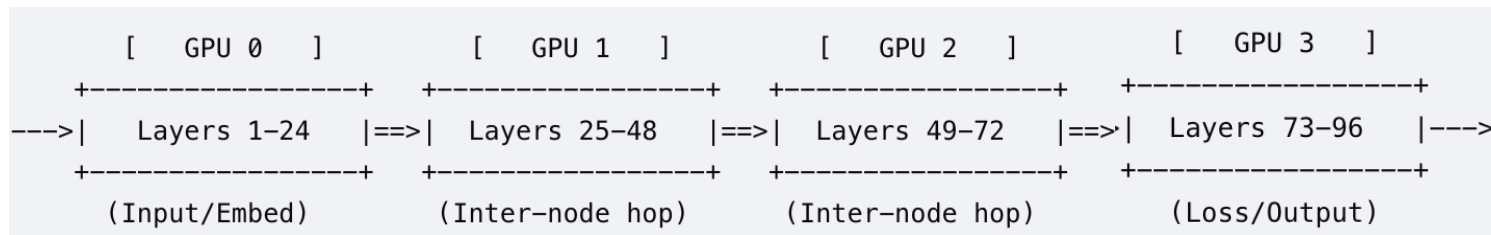
row parallelism



- **Column Parallelism:** Splits the weight matrix column-wise. Each GPU computes a partial output. No communication is needed—outputs are simply concatenated.
- **Row Parallelism:** Splits the weight matrix row-wise. Each GPU computes a partial result. An **all-reduce** operation sums the partial results across all GPUs.
- **Bandwidth Scaling:** With Tensor Parallelism=8 on H100s, effective memory bandwidth scales to ~26.8 TB/s (8×3.35 TB/s). This reduces decode latency by ~8x.

Hard Requirement: All-reduce requires NVLink (~900 GB/s on H100) within single node. Tensor parallelism across nodes is too slow for live serving.

Parallelism: Pipeline parallelism

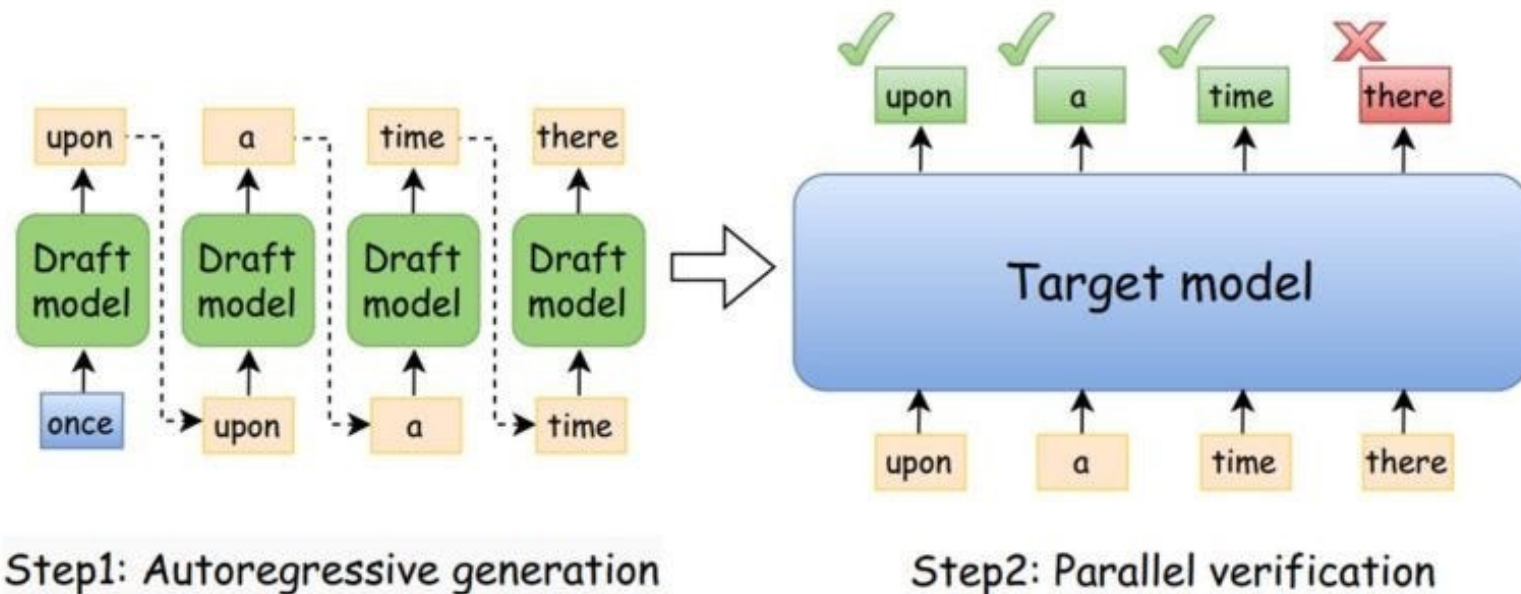


- **Layer Splitting:** GPU 0 handles layers 1–24, GPU 1 handles layers 25–48 etc.. Activations flow sequentially from one stage to the next.
- **Communication:** Unlike Tensor Parallelism which transmits massive data matrices constantly, Pipeline Parallelism only needs to send the final activation tensors to next GPU and tolerates **InfiniBand** latency (100GB across node v.s. 1.8TB within node).
- **The Cost:** Pipeline bubbles. Earlier GPUs sit idle while later GPUs process activations. The bubble fraction is approximately $(N-1)/N$ for a naive pipeline.

Production Standard: Hybrid strategy. TP=8 within a node (exploiting NVLink) + PP=4 across nodes (tolerating InfiniBand).

Speculative Decoding

Uses a draft model to verify multiple tokens in one pass, yielding **2–3× decode speedup**



Research indicates the sweet spot for draft model sizing is roughly **1/8th to 1/12th the size of the target model**.

Tokenizer



Embedding



Prefill

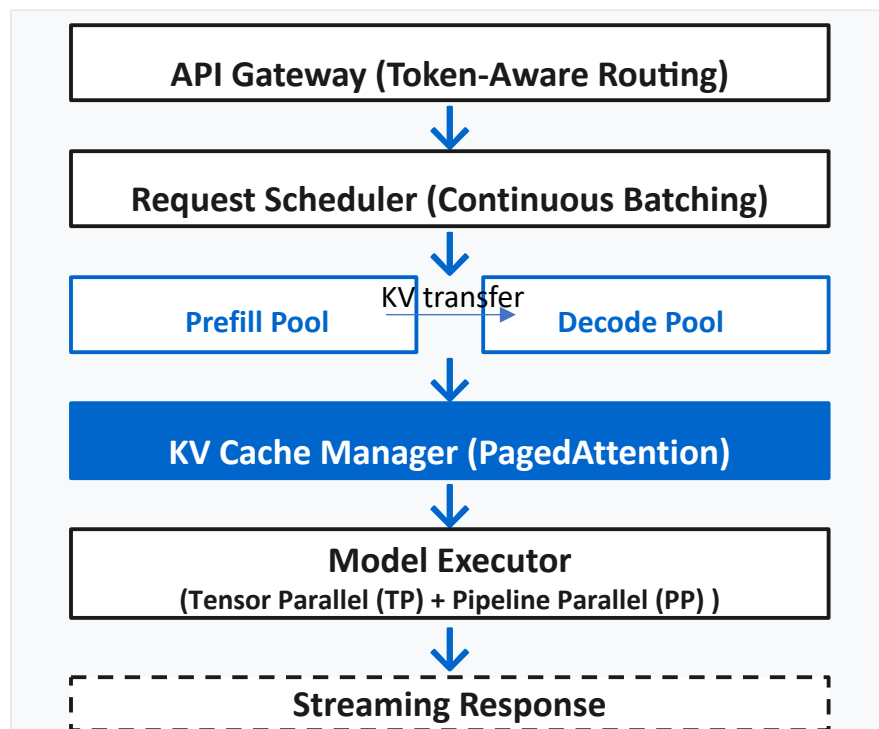


Decode



Detokenize

A production LLM inference engine integrates all these components into one coherent system



THE CENTRAL INSIGHT

Every component exists because LLM inference is **stateful**, **variable-length**, and **resource-heterogeneous**.

- **Stateful:** KV Cache grows for the entire request lifetime, requiring PagedAttention.
- **Variable-length:** Output length is unpredictable, requiring Continuous Batching.
- **Resource-heterogeneous:** Prefill and Decode have different bottlenecks, requiring Disaggregation.

Six Things Every GenAI Practitioner Should Know

1 Prefill is Compute-Bound. Decode is Memory-Bound.

Prefill dominates TTFT. Decode dominates ITL.

3 Continuous Batching is the Baseline.

Static batching is obsolete for autoregressive generation. Schedule at the token step.

5 Tensor Parallelism Scales Decode Bandwidth.

It scales linearly with GPU count to reduce latency. NVLink is a hard requirement.

2 The KV Cache Dominates GPU Memory.

PagedAttention solves fragmentation with virtual memory paging, maximizing concurrency.

4 Prefill/Decode disaggregation

Chunked prefill or prefill/decode disaggregation is required to maintain strict streaming SLOs.

6 Speculative Decoding Recovers Idle Compute.

It accelerates decoding by 2x to 3x reduction in latency and ~20% cost reduction, but is only effective at low concurrency when GPU is memory-bound.